EC-350 AI and Decision Support Systems

Week 16
Convolutional Neural Networks

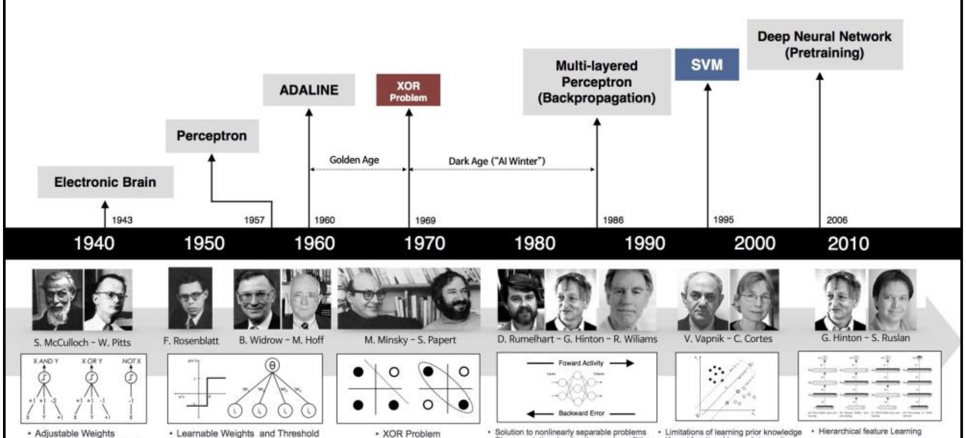
Dr. Arslan Shaukat

NUST

Acknowledgements: Lecture slides material from
Dr Rizwan A Khan

1

History



2

Fathers of Neural Network/Deep Learning

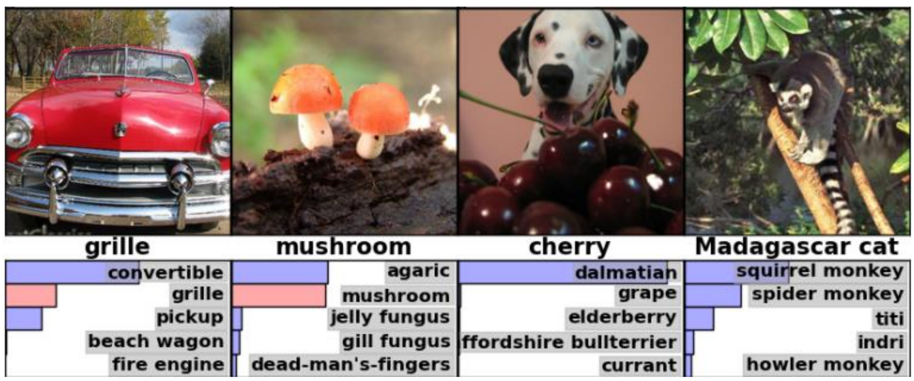
- Turing Award 2018: Geoffrey Hinton, Yoshua Bengio and Yann LeCun for their work on deep learning.
- These three are referred to by some as the "Godfathers of AI" and "Godfathers of Deep Learning".

Breakthrough in Deep Learning

- TorontoNet/AlexNet - Geoff Hinton 2012 - 60M trainable parameter - 1000 classes

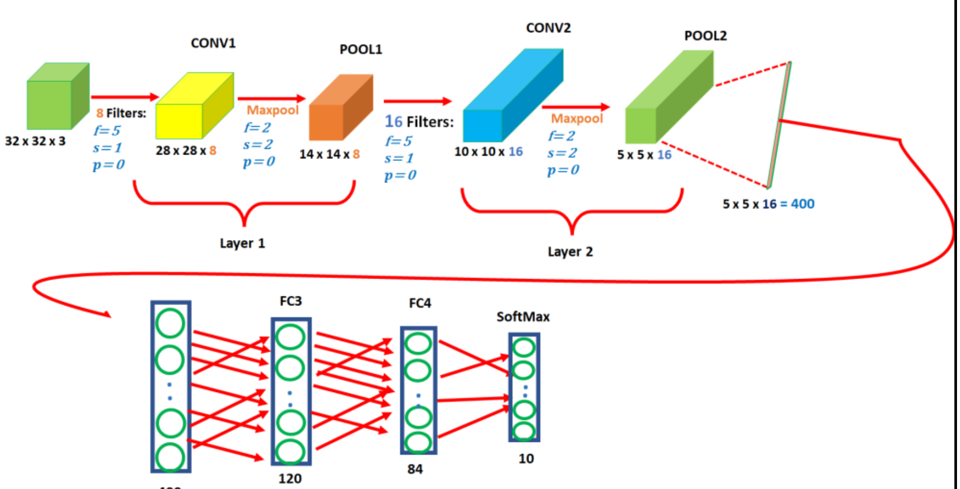
Breakthrough in Deep Learning

- TorontoNet/AlexNet - Geoff Hinton 2012 - 60M trainable parameter - 1000 classes



5

CNN



6

Convolution

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

Image Filter Output

Convolution

3 1	0 0	1 -1	2	7	4
1 1	5 0	8 -1	9	3	1
2 1	7 0	2 -1	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5			

Image Filter Output

Convolution

3	0 1	1 0	2 -1	7	4
1	5 1	8 0	9 -1	3	1
2	7 1	2 0	5 -1	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4		

Image Filter Output

Convolution

3	0	1	2	7	4
1 1	5 0	8 -1	9	3	1
2 1	7 0	2 -1	5	1	3
0 1	1 0	3 -1	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10			

Image Filter Output

Convolution

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1 1	7 0	8 -1
4	2	1	6 1	2 0	8 -1
2	4	5	2 1	3 0	9 -1

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Image Filter Output

Learning Edge Detection

- Gives more weight to pixel values adjacent to center pixel.

1	0	-1
1	0	-1
1	0	-1

1	0	-1
2	0	-2
1	0	-1

Sobel Filter

Learning Edge Detection

- There is a lot of debate which filter works better and different researchers have proposed many other filters for the same purpose.

1	0	-1
1	0	-1
1	0	-1

1	0	-1
2	0	-2
1	0	-1

Sobel Filter

3	0	-3
10	0	-10
3	0	-3

Scharr Filter

- These are just vertical edge detectors. We could run these filters in different orientations i.e. 45, 60, 75 degrees etc.

Learning Filter Weights

- In deep learning, it is common to learn these filters by considering kernel/filter values as weights/parameters of the model to detect edges in complex (clutters, occlusion, lighting conditions) images.
- It is impossible to handcraft edge detector that works well for all types of images with varying level of complexity.
- Model can learn to detect edges at various orientations i.e. multiple filters can be learned.
- By doing so Neural Nets can robustly learn low level features which is not possible with handcrafted filters.

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Learning Filter Weights

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

I/P Image

*

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Filter

=

O/P Image

Image size after Convolution

- Size of output image is different as for filter of size 3 × 3 when applied on image of size 6 × 6, there are only 4 × 4 possible positions where this filter fits in.
- Usually filter is not applied on the boundary pixels.

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Image size calculation

- Let's suppose:
  - *Input image size is n × n*
  - *Filter size is f × f*
  - *Output image size will be n-f+1 × n-f+1.*
  - *In example: 6 - 3 + 1 × 6 - 3 + 1 = 4 × 4*

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Issues

- Shrinking output: Every time when image is convolved with the filter, its size shrinks. It means only few times this operation can be applied before image shrinks to the level that it losses information (problem when you build deep CNN).
- Information thrown away: Contribution of boundary pixels (in an input image) is less as compared to other pixels in the calculated output image (corner boundary pixel only used once).

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Padding

- Solution for both the problem highlighted in previous slide is to “pad” the image before applying convolution.
- After applying padding, the output image size remains 6 × 6, same as original input image size (before padding).

0	0	0	0	0	0	0	0
0	3	0	1	2	7	4	0
0	1	5	8	9	3	1	0
0	2	7	2	5	1	3	0
0	0	1	3	1	7	8	0
0	4	2	1	6	2	8	0
0	2	4	5	2	3	9	0
0	0	0	0	0	0	0	0

8 x 8

*

1	0	-1
1	0	-1
1	0	-1

3 x 3

=

6 x 6

Strided Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product → 3 -1

6 x 6 image

Strided Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

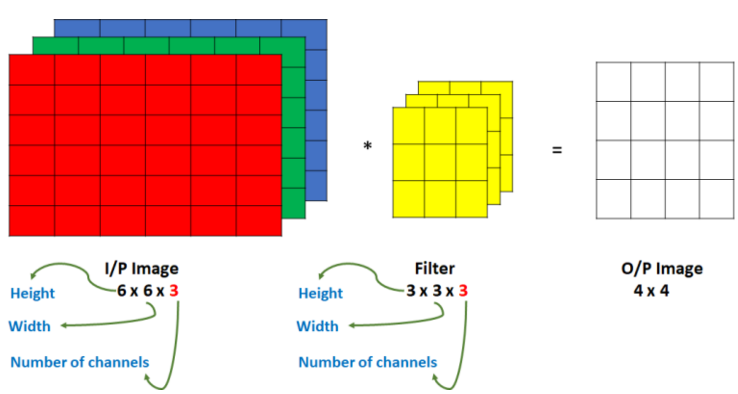
6 x 6 image

3 -3

21

Convolution over Volumes

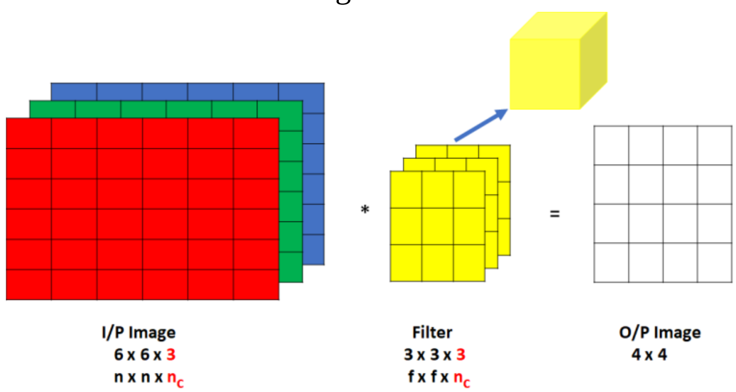
- Input image has three channels (3D volume / image) i.e. red, green and blue.



22

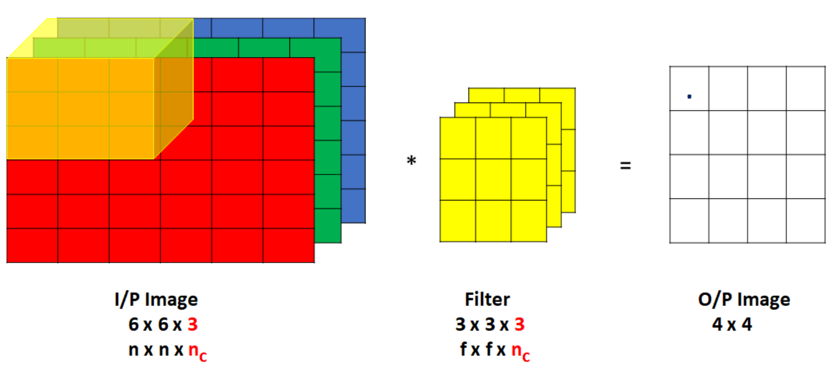
Convolution over Volumes

- We are applying filter(s) to extract discriminative features.
- Number of channels (n_c) are same (must match) in input image and filter.
- Dimensions of resultant image will be 4×4.



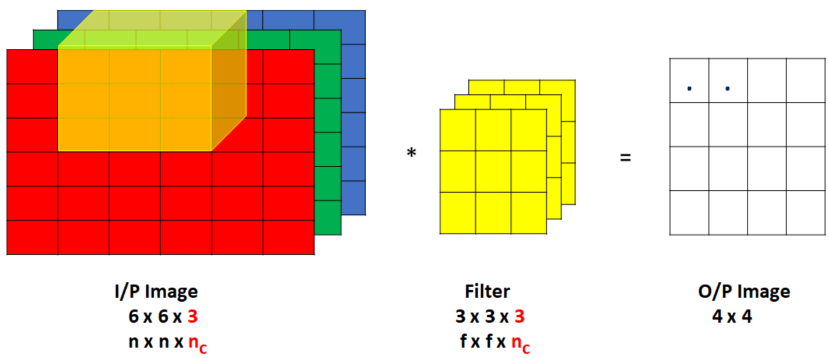
Convolution over Volumes

- To compute the output, filter 3D cube is to placed over image (upper left position) and convolution is performed.
- Notice that filter 3D cube has now 27 numbers / parameters / weights.



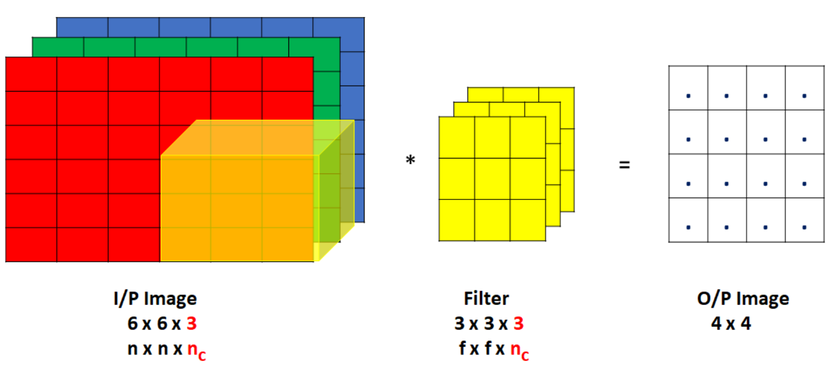
Convolution over Volumes

- To compute output pixel values, 27 numbers / parameters of filter are multiplied with corresponding 27 pixel values of input image's red, green and blue channels (9 pixel values from each channel).
- Then finally, the values are summed.



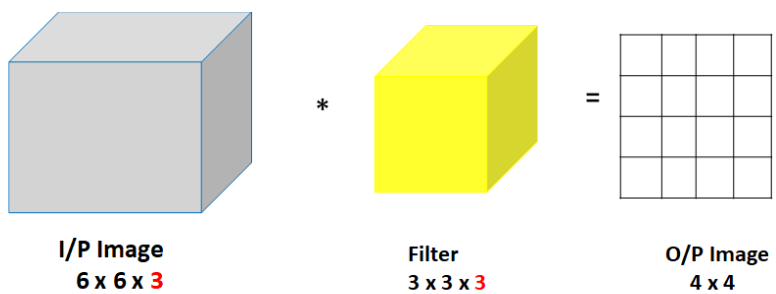
Convolution over Volumes

- Continue sliding filter cube over image cube till the possible image locations and continue calculation output pixel values



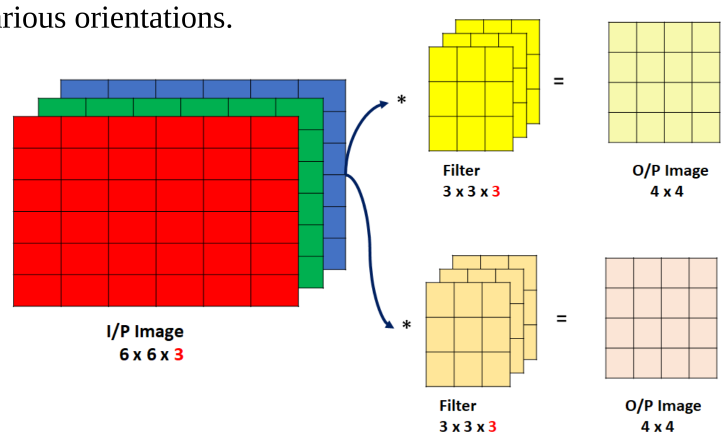
Summary: Convolution over Volumes

- Neural Net (CNN) can learn filters that extract information related to the given task.
- Filters are not hard coded. CNN can learn any filter i.e. vertical edge detector, horizontal or diagonal edge detectors or edges at 70, 35, 60 degrees etc, or it can learn to detect edge only in one channel.



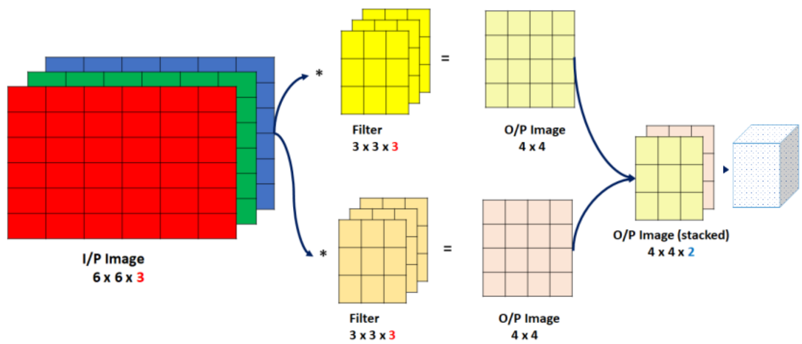
Multiple Filters

- Another building block that is important to understand CNN, is what happens if multiple filters are applied at the same time to extract robust information. For example, extracting edges at various orientations.



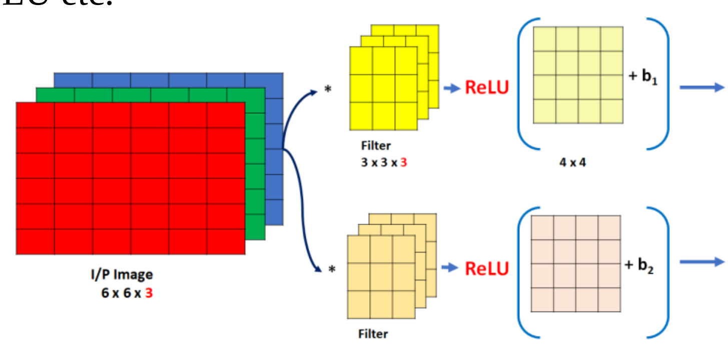
Multiple Filters

- Output from each filter convolution is shown in different color. Then output after convolving image with each filter is stacked to get the final output (output in the form of volume). The value “2” is the number of channels in the output volume / cube and this number corresponds to numbers of filters applied.

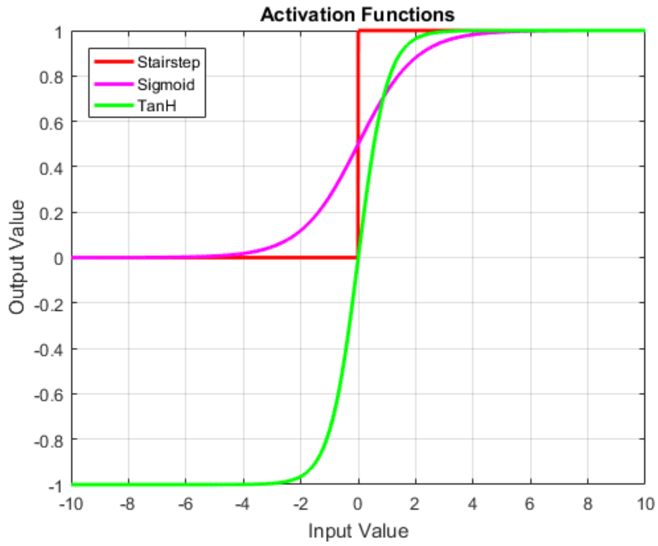


One Layer of ConvNet

- In ConvNet filter parameters are model parameters or weights. Like in NeuralNet, here these learned weights (W or θ) along with “bias” (added to convolution output) are passed to non-linear function (activation function) i.e. ReLU etc.

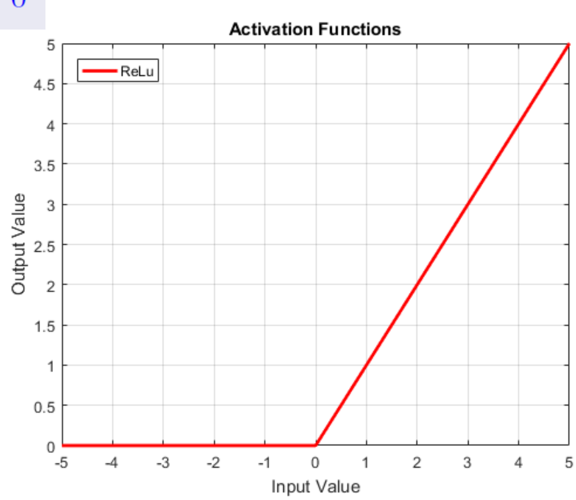


Activation Functions



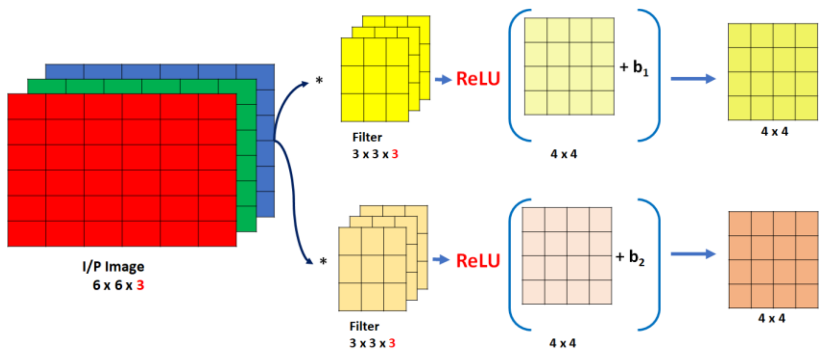
Activation Functions

$$f(x) = \begin{cases} 0 \;\; for \;\; x \leq 0 \\ x \;\; for \;\; x > 0 \end{cases}$$



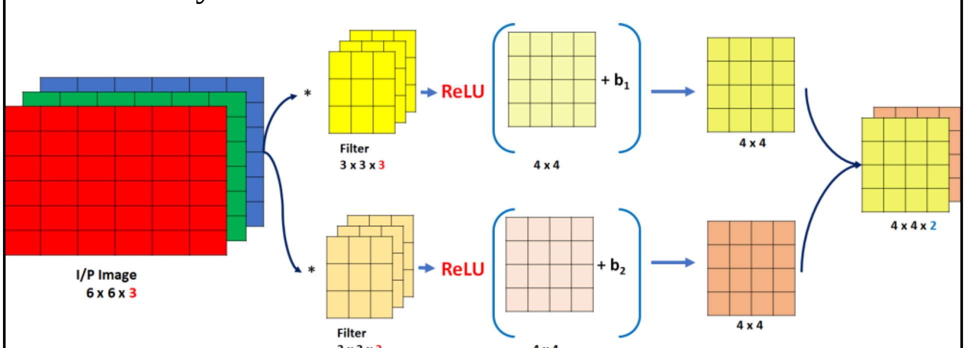
One Layer of ConvNet

- Output after applying bias and activation (non-linearity) function gives 4 × 4 output, sometimes called activation or feature map.



One Layer of ConvNet

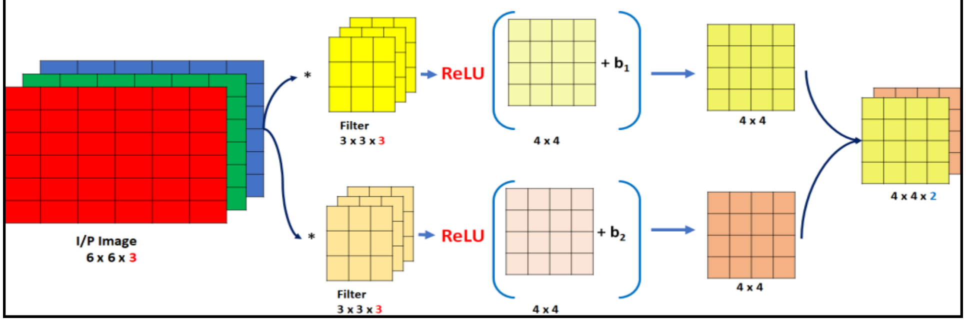
- As seen earlier, these feature maps are stacked to obtain final output of 4 × 4 × 2. This process, where initial input image was 6 × 6 × 3 and output was 4 × 4 × 2 comprises of one layer of ConvNet.



One Layer of ConvNet

- To see this ConvNet layer from the lens of NeuralNet, the forward prop equation is:

  z[1] = w[1]a[0] + b[1], a[1] = g(z[1])
- In this case g is ReLu activation function, filter weights are w and a[0] is image pixel values. w[1]a[0] is just a convolution / linear operation (weighted sum).
- a[1] (cube of 4 × 4 × 2) is activation for next layer

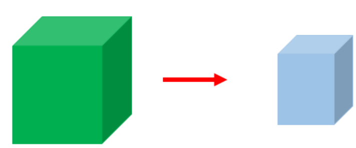


35

One Layer of ConvNet

- To see this ConvNet layer from the lens of NeuralNet, the forward prop equation is:

  z[1] = w[1]a[0] + b[1], a[1] = g(z[1])
- In this case g is ReLu activation function, filter weights are w and a[0] is image pixel values. w[1]a[0] is just a convolution / linear operation (weighted sum).
- a[1] (cube of 4 × 4 × 2) is activation for next layer



I/P Image
6 x 6 x 3

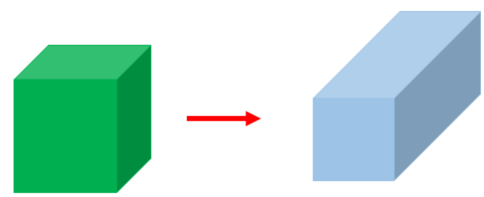
4 x 4 x 2

$a^{[0]}$ $a^{[1]}$

36

One Layer of ConvNet

- If there were 10 filters, then:



I/P Image
6 x 6 x 3

4 x 4 x 10

$a^{[0]}$ $a^{[1]}$

One Layer of ConvNet

- Calculate total parameters of ConvNet layer. Layer has 10 filters of size $3 \times 3 \times 3$.
- It will have:

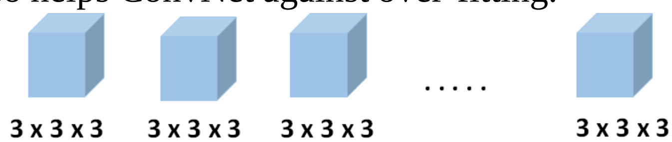
$$(27 + 1(\text{for bias})) \times 10 = 280 \text{ parameters.}$$

3 x 3 x 3 3 x 3 x 3 3 x 3 x 3 . . . . . 3 x 3 x 3

Parameters: (3 x 3 x 3) + 1 for one filter
27 + 1 = 28

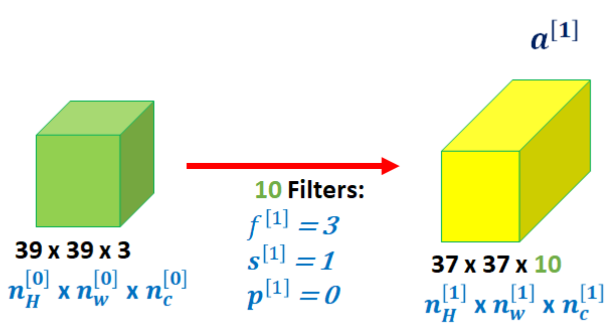
One Layer of ConvNet

- Notice one nice property, no matter how big the input image is, for example 1000 × 1000 × 3, the number of parameters to learn remains fixed as same filter slides across the image.
- In NeuralNets, number of parameters usually grows with the size of an input, thus sometimes its restrictive.
- This also helps ConvNet against over-fitting.



Parameters: (3 x 3 x 3) + 1 for one filter
27 + 1 = 28

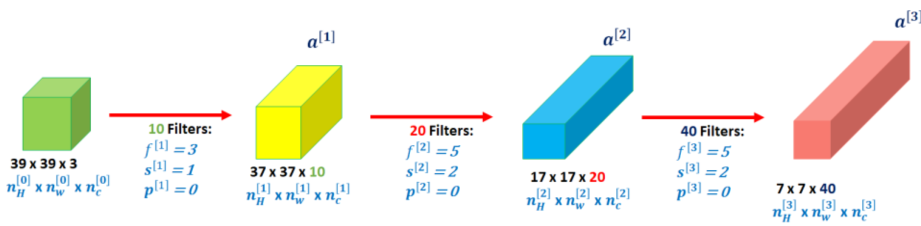
Example of Deep ConvNet



Example of Deep ConvNet

- Notice after using stride of two, output dimensions have shrunk faster.
- So after applying three convolution layers (with parameters shown), an image of size 39 × 39 × 3 is transformed to 7 × 7 × 40 features.

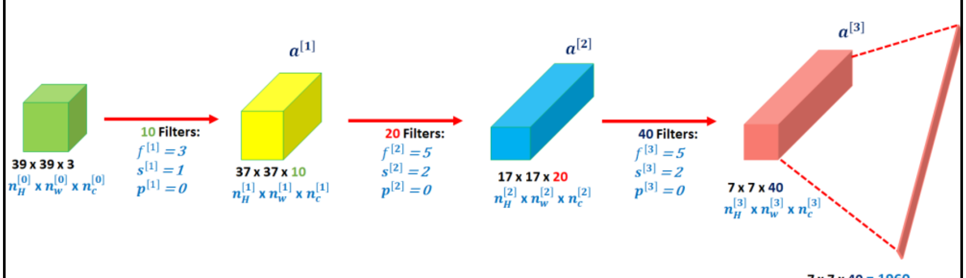
Formula for calculating:
[(input img size - filter size + 2*padding) / stride] + 1
((37-5+0)/2)+1=17



41

Example of Deep ConvNet

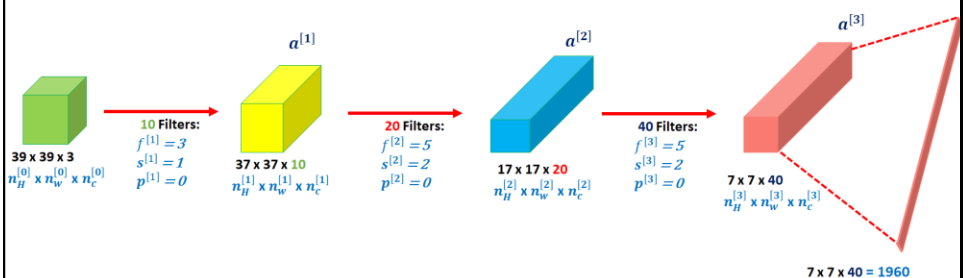
- Finally, whats commonly done is to take 7 × 7 × 40 or 1960 features volume and flatten it into 1960 units vector.
- For classification this 1960 units vector is fed to "Softmax" unit.



42

Example of Deep ConvNet

- Lot of work in designing CNN is to select hyper-parameters for the model i.e. number of layers, size and number of filter, type of convolution, size of stride etc. Few heuristic on this will be discussed later.
- Take away: as we go deeper, generally, size of image / activation gradually decreases, whereas number of channels generally increase.

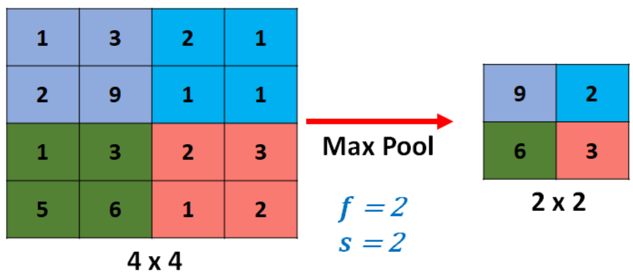


Other Layers

- There are few other types of layers, apart from convolutional layer, that is yet to be discussed. These are relatively simpler to understand.
  - *Convolutional layer (CONV)*
  - *Pooling layer (POOL)*
  - *Fully connected layer (FC)*

Pooling Layer: Max Pooling

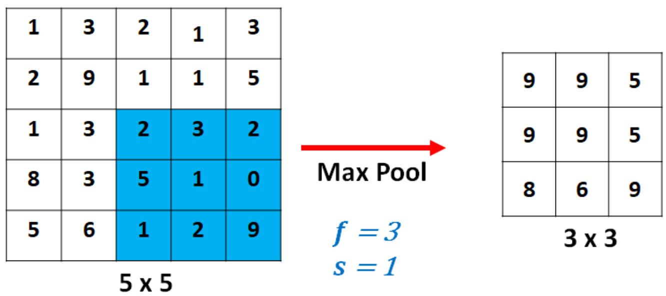
- Extracted max value from 2 × 2 non-overlapping regions.
- This is as if, applying a filter of size f = 2 with stride length of s = 2. These are hyper parameters of this layer.



Pooling Layer

- A limitation of the feature map output of convolutional layers is that they record the precise position of features in the input (not invariant to movement of region of interest). This problem in signal processing domain is usually dealt with down sampling signal, that contains large or important elements without much of fine details. In ConvNet this is achieved by Pooling layer.
- Pooling layers reduce the dimensions of the feature maps. By doing so:
  - *Reduces the number of parameters to learn.*
  - *Reduces the amount of computation performed in the network.*
- There are couple of hyper-parameters to select but it has no learnable parameter

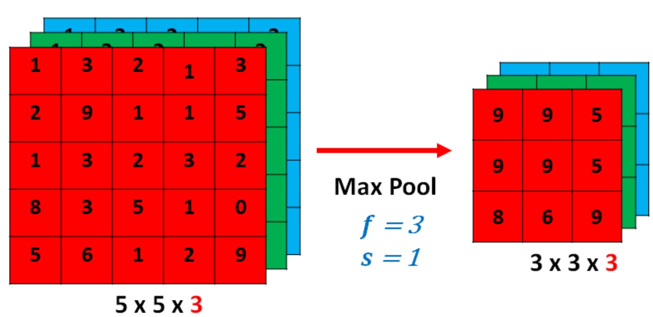
Pooling Layer: Max Pooling



47

Pooling Layer: Max Pooling

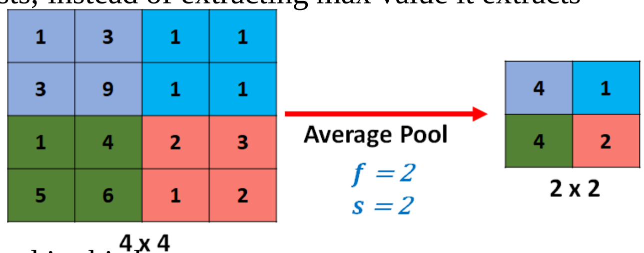
- If we have 3D input and we apply max pooling
- In this example output will be 3×3×3 (number of channels of output is same as number of channel of input image / feature map).
- Max pooling operations are applied on each channel independently.



48

Pooling Layer: Average Pooling

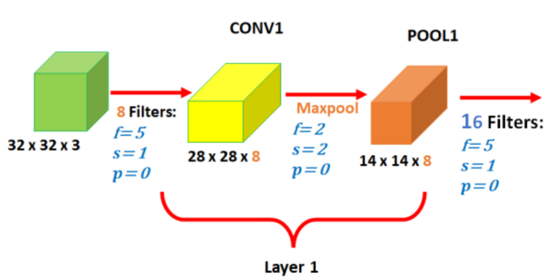
- There is another type of pooling that is not used very often, it is "average pooling".
- As the name suggests, instead of extracting max value it extracts average value.



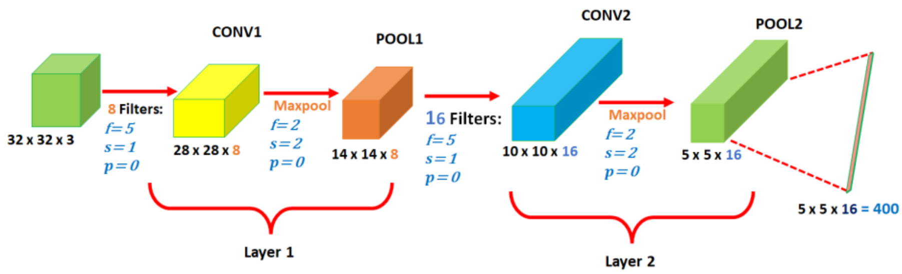
- Padding is rarely used in this layer.
- Number of output channels are same as input channels.
- There are couple of hyper-parameters (f : filter size, s : stride and type of pooling: max or average) to select but it has no learnable parameter

CNN Example

- Generally in literature, group of convolution and pooling layers are called as "Layer".
- This is due to the fact that pooling layer has no trainable parameters /weights. Thus, total number of layers with trainable parameters are counted as layer.
- Sometimes this counting heuristic is not held and all layers are counted in the network.

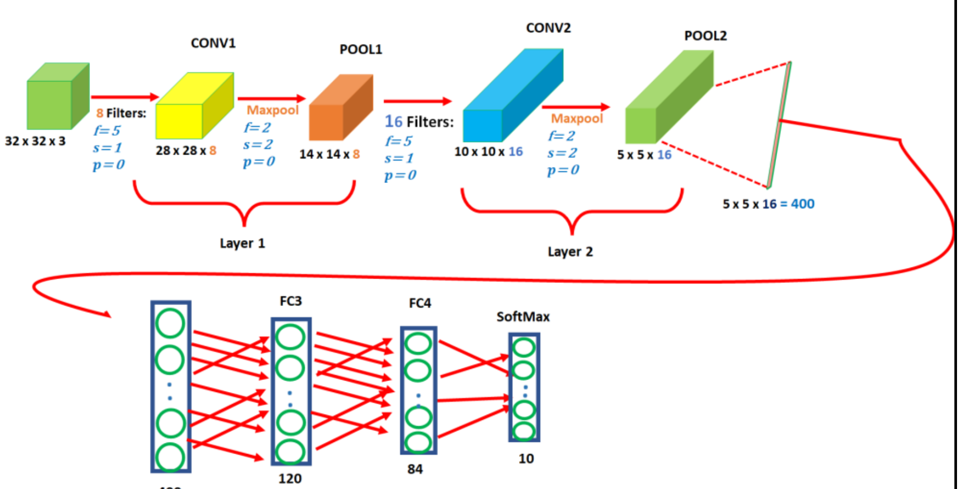


CNN Example



51

CNN Example: Fully Connected Layer



52

FC Layer

- In the last of ConvNet, output is flattened ($5 \times 5 \times 16 = 400$) and fed to fully connected layer(s) before applying "Softmax" unit .
- FC layer is just like layer of neural network (standard NeuralNet). In FC layer all the neurons in layer "l" are connected to neuron in the next layer "$l + 1$" (densely connected).
- Last "Softmax" unit has 10 neuron. This shows that it's 10-class classification problem.

CNN Example: Parameters

- One thing to notice is that number of trainable parameters of CONV layers are much less than trainable parameters of FC layers.

[(filter size * filter size * input depth) * number of filters] + number of filters

Calculate number of trainable parameters

	Activation Shape	Activation Size	Number of Parameters
Input	$32 \times 32 \times 3$	$32 \times 32 \times 3 = 3072$	0
CONV1, 8 filters with $f = 5, s = 1$	$28 \times 28 \times 8$	$28 \times 28 \times 8 = 6272$	$((5 \times 5 \times 3) \times 8) + 8 = 608$
POOL1 with $f = 2, s = 2$	$14 \times 14 \times 8$	$14 \times 14 \times 8 = 1568$	0
CONV2, 16 filters with $f = 5, s = 1$	$10 \times 10 \times 16$	$10 \times 10 \times 16 = 1600$	$((5 \times 5 \times 8) \times 16) + 16 = 3216$
POOL2 with $f = 2, s = 2$	$5 \times 5 \times 16$	$5 \times 5 \times 16 = 400$	0
FC3	(120 , 1)	120	$(400 \times 120) + 120 = 48120$
FC4	(84 , 1)	84	$(120 \times 84) + 84 = 10164$
Softmax	(10 , 1)	10	$(84 \times 10) + 10 = 850$

State of the Art Networks

- LeNet
- AlexNet
- VGG
- DenseNet
- ResNet
- Inception